

Spring Boot

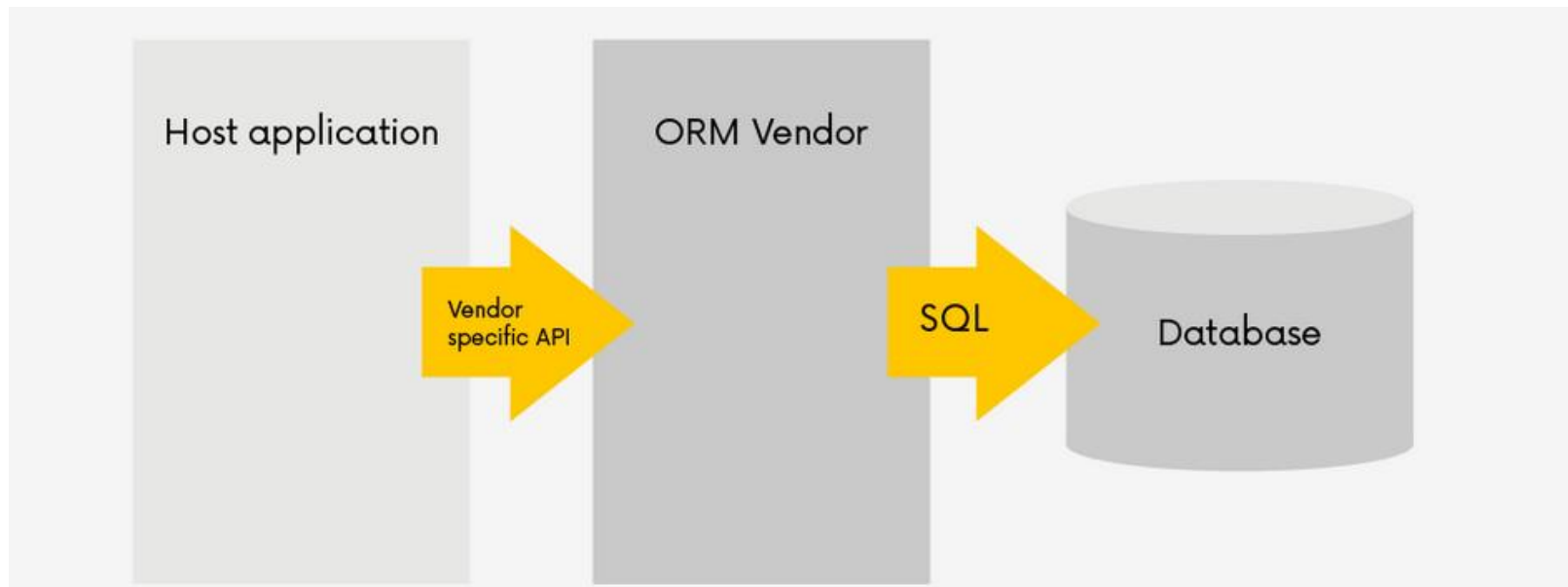
Introduction à la JPA

Introduction

- Spring Boot (SB) offre de nombreux outils et bibliothèques prédéfinis qui simplifient le développement d'applications.
- Il offre une approche modulaire et flexible du développement d'applications et simplifie des tâches telles que l'injection de dépendances, la programmation orientée aspects et la gestion des transactions.
- La Programmation Orientée Objet (POO) s'est imposée dans tous les domaines et notamment dans le développement Web en forte relation avec les bases de données relationnelles.
- L'un des composants Spring les plus utiles est le Spring Data Framework, **basé sur JPA**.
- Son principal avantage est qu'il simplifie le travail des développeurs qui utilisent fréquemment **l'ORM (Object Relational Mapping)** pour accéder aux bases de données dans leurs projets, car il n'est presque plus nécessaire d'écrire du code d'accès ou des requêtes de base de données.

C'est quoi JPA ?

- JPA signifie Java Persistence API et est une spécification standard pour l'accès, la gestion et le stockage de données entre des objets Java et des bases de données relationnelles.
- JPA est une technologie populaire utilisée pour l'utilisation ORM (Object-Relational Mapping) basée sur Java qui permet aux applications Java d'interagir de manière transparente avec les bases de données relationnelles.
- Elle offre **une API orientée objet pour les opérations de base de données**, ce qui réduit le besoin de requêtes SQL de bas niveau.



Quelles fonctions assurées par JPA ?

L'utilisation de Spring Boot et de la JPA dans le développement d'applications Web doit permettre de simplifier les opérations CRUD (Create Read Update Delete) :

- Insérer les données dans une BDD
- Extraire les données de la BDD
- Mettre à jour les données de la BDD
- Supprimer les données de la base.

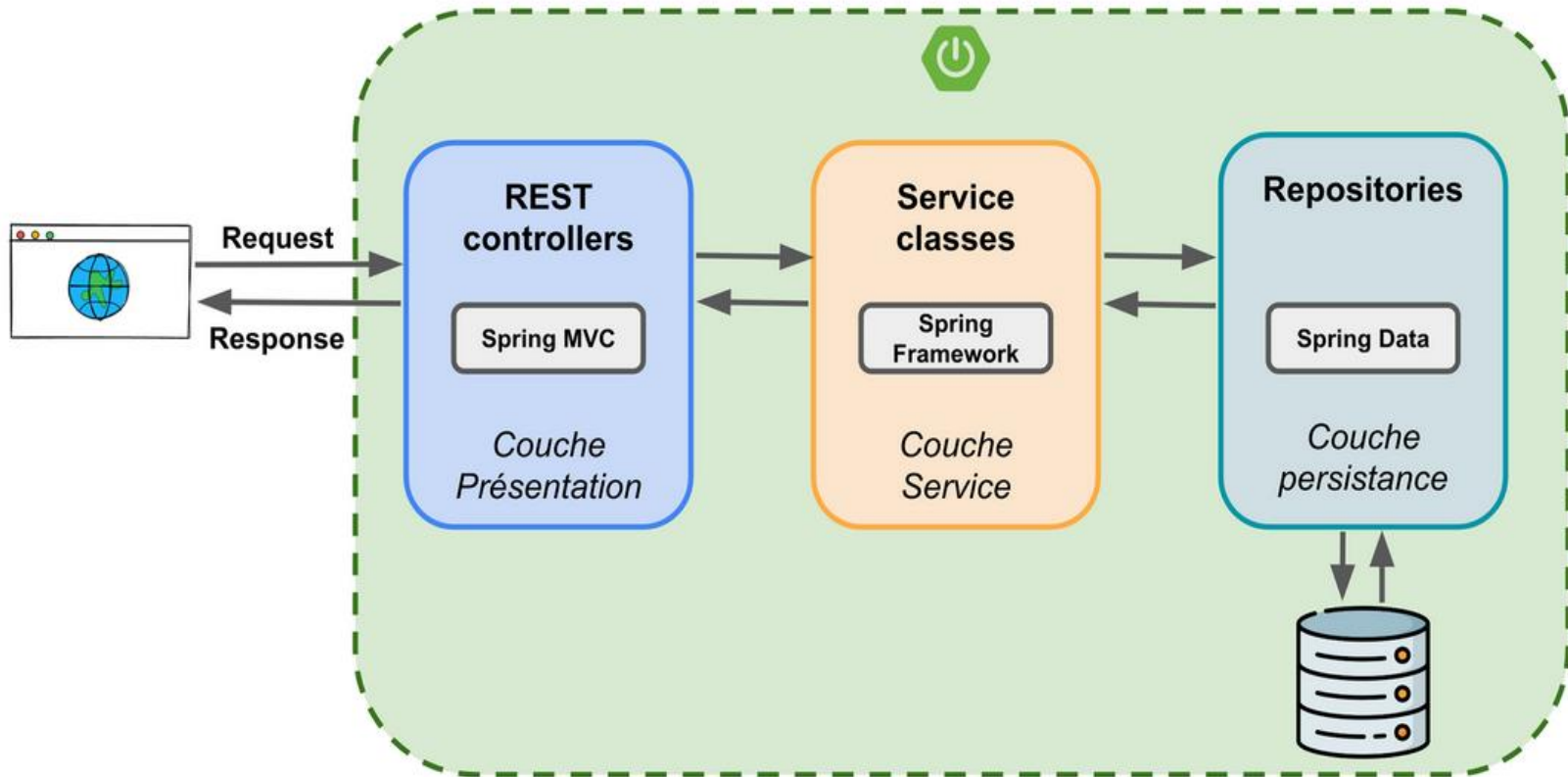
L'accès aux BDD dans SB se fait principalement via des API, en évitant au maximum l'utilisation des requêtes SQL.



Architecture fonctionnelle

L'architecture fonctionnelle de SB s'appuie sur les trois principales couches (Layers) :

- Controller
- Model
- View



Etapes de mise en œuvre

1. Configuration des dépendances dans le fichier POM.XML
2. Création de la base de données (Avec Xampp, ...)
3. Configuration du fichier : **src/main/resources/application.properties**
4. Création des entités (Entity) JPA
5. Création de l'interface Repository
6. Création de la couche Service
7. Création du contrôleur REST
8. Lancement de l'application
9. Définition des URL de votre API

Configuration des dépendances dans POM.XML

Vérifier la présence des dépendances suivantes dans le fichier POM.XML

```
<dependencies>
  <dependency>
    <groupId>org.springframework.data</groupId>
    <artifactId>spring-data-jpa</artifactId>
  </dependency>
</dependencies>
```

```
  <dependency>
    <groupId>com.mysql</groupId>
    <artifactId>mysql-connector-j</artifactId>
    <scope>runtime</scope>
  </dependency>
```

Configuration du fichier : *application.properties*

Pour utiliser une base MySQL, il faut ajouter au fichier :

src/main/resources/application.properties

```
spring.datasource.url=jdbc:mysql://localhost:3306/stock  
spring.datasource.username=root  
spring.datasource.password=  
spring.jpa.generate-ddl=true  
spring.jpa.hibernate.ddl-auto=update
```

Création de la base de données

Sous MySQL (Xampp), créer une base de données (Juste le nom de la base)

Création des entités

Une entité est classe (au sens POO), qui permet de représenter les colonnes d'une table de la base de données.

L'annotation @Entity permet à JPA de **créer une table dans la base** à partir de la structure de votre entité.

```
@Entity 12 usages
public class Article {

    @Id 1 usage
    @GeneratedValue(strategy = GenerationType.AUTO)
    private Long id;
    private String nom; 3 usages
    private String categorie; 3 usages
    private double prixUni; 3 usages
    private int qte; 3 usages

    public Article(String nom, String categorie, double prixUni, int qte) {
        this.nom = nom;
        this.categorie = categorie;
        this.prixUni = prixUni;
        this.qte = qte;
    }

    public Article() { no usages
    }

    public Long getId() { no usages
        return id;
    }
}
```

Création des entités

```
public String getNom() { no usages
    return nom;
}

public String getCategorye() { no usages
    return categorie;
}

public double getPrixUni() { no usages
    return this.prixUni;
}

public void setCategorye(String categorie) { no usages
    this.categorie = categorie;
}

public void setPrixUni(double prixUni) { no usages
    this.prixUni = prixUni;
}

public void setQte(int qte) { no usages
    this.qte = qte;
}
```

Création des entités

```
    public void setNom(String nom) { no usages  
        this.nom = nom;  
    }  
  
    public int getQte() { no usages  
        return qte;  
    }  
} //Fin de la classe Entity
```

Création du Repository

- *Spring boot* s'organise autour de la notion de *repository*.
- SB fournit une interface `CrudRepository`
- Le Repository fournit une interface marqueur générique [`Repository<T, ID>`](#).
- Le type `T` correspond au type de l'objet géré par le *repository*.
- Le type `ID` correspond au type de la clé d'un objet.

```
import mmi.td.stock.model.Article;
import org.springframework.data.repository.CrudRepository;
import org.springframework.stereotype.Repository;

@Repository 2 usages
public interface ArticleRepository extends CrudRepository<Article, Long> {
}
```

La couche Service

- Cette couche permet d'implémenter la logique métier de l'application.
- Pour réaliser les opérations sur la base, le service doit utiliser le Repository.

```
@Service 2 usages
public class ArticleService {

    @Autowired 4 usages
    private ArticleRepository articleRepository;

    public Article saveArticle(Article article) { 1 usage
    |     return articleRepository.save(article);
    |
    |
    |
    }

    public Iterable<Article> getAllArticles() { 1 usage
    |     return articleRepository.findAll();
    |
    |
    |
    }

    public Article getArticleById(Long id) { 1 usage
    |     return articleRepository.findById(id).orElse( other: null);
    |
    |
    |
    }

    public void deleteArticleById(Long id) { 1 usage
    |     articleRepository.deleteById(id);
    |
    |
    |
    }
}
```

Les contrôleurs

- Le contrôleur permet de transmettre les données traitées à la View.
- Le REST contrôleur utilise les services, par injection des dépendances

```
@RestController no usages
@RequestMapping("/api/articles")
public class ArticleController {

    @Autowired 4 usages
    private ArticleService articleService;

    @PostMapping no usages
    public Article createArticle(@RequestBody Article article) {
        return articleService.saveArticle(article);
    }

    @GetMapping no usages
    public Iterable<Article> getAllArticles() {
        return articleService.getAllArticles();
    }

    @GetMapping("/{id}") no usages
    public Article getArticleById(@PathVariable Long id) {
        return articleService.getArticleById(id);
    }

    @DeleteMapping("/{id}") no usages
    public void deleteArticle(@PathVariable Long id) {
        articleService.deleteArticleById(id);
    }
}
```

Lancement de l'application & Utilisation de l'API

npm run dev

Création d'un nouvel utilisateur

POST /api/articles

Content-Type: application/json

```
{  
  "nom": "Microphone Pro",  
  "categorie": "Dispositif interactif - DI",  
  "prixUni": 75.00,  
  "qte": 15  
}
```

Afficher tous les articles

GET /api/articles

Afficher l'utilisateur par ID

GET /api/articles/{id}

Supprimer un utilisateur

DELETE /api/articles/{id}

TD d'application

1. Créer un projet SB nommé stock, via <https://start.spring.io>
2. Intégrer les dépendances : JPA, MySQL, DEV WEB, DEV Tools
3. Créer une base de données nommée : **stock** (En utilisant Xampp)
4. Configurer votre fichier **application.properties**
5. Créer l'entité Article (id, nom, categorie, prixUni, qte). Prévoir les setters et les getters nécessaires de la classe.
6. Créer le Repository, en vous référant du slide 12
7. Créer la classe service en vous référant au slide 13
8. Créer les REST Controller, en vous appuyant sur le slide 14
9. Démarrer votre application et tester votre API, en utilisant postman.